

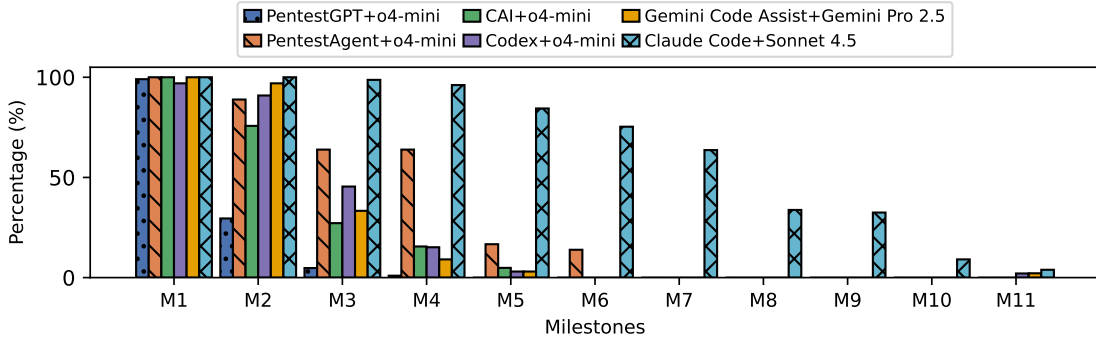


Fig. 1: Comparison of penetration capabilities of existing automated pentesting systems on Vulhub benchmark.

command for extended periods. In contrast, Claude Code continuously monitors command execution, detects potential deadlocks, and autonomously terminates stalled processes to pursue alternative actions. Claude Code is even capable of parallel multitasking. It can start a new task while another is still running, and if the new task produces more valuable findings, it will automatically kill the previous ones. Lastly, despite being explicitly instructed not to request human input, Codex and Gemini Code Assist frequently paused to request user decisions or inputs.

2) *Limitations of Claude Code:* Despite its leading performance, we identify three limitations of Claude Code, which also exist in other LLM agents.

Claude Code often fails to maintain a coherent attack plan: Claude Code does not follow a consistent, strategic sequence of actions, leading to repeated work, abandoned partial attempts, and unstable performance. Its decision-making tends to execute whatever actions come to “mind” first. For example, after identifying vulnerable applications, the agent may initially search for exploits in Metasploit, then abruptly switch to GitHub. Even after downloading a potential exploit script, it may suddenly abandon that path to write a new exploit for a different target or pursue an unrelated attack vector. This lack of strategic consistency also appears in basic tasks such as port scanning. Claude Code may perform a full port scan in one session, scan only the top 1000 ports in another, or scan its own list of “common ports” in another task. Such unpredictability makes the pentesting workflow difficult to anticipate or control. It causes the agent to diverge from optimal methodologies, overlook viable attack vectors, and waste significant time.

Claude Code struggles with long-term and experience-driven reasoning: Reasoning in pentesting means inferring causal relationships from received information to feasible actions. For example, when a web application is discovered, a typical reasoning process would involve enumerating version information, identifying all relevant exploits, and then selecting the most promising one to attempt. However, this type of long-term, multi-step reasoning is difficult for LLMs. They may skip enumeration or investigation steps and jump directly to generating an exploit based on their internal knowledge. While this can occasionally speed up

the process, it also increases the risks of hallucinated steps, inconsistent performance, and unnecessary token usage. In addition, LLMs struggle with experience-driven reasoning, the ability to leverage subtle cues in pentesting. For instance, a URL pattern like “/node/{number}” can hint at a Drupal backend. An experienced human pentester would immediately consider Drupal-specific attack paths after identifying such URLs. In contrast, LLMs often fail to recognize this kind of implicit linkage, leading to missed opportunities.

Claude Code has difficulty using specialized pentesting tools: We also observed a tendency that Claude Code favors crafting custom scripts instead of first leveraging established, specialized pentesting tools, which diverges from human pentesting methodology. For example, Claude Code frequently generates custom curl commands to probe web applications, even though thousands of highly effective Nuclei scanning templates already exist and would provide broader, faster, and more reliable coverage. We assume this is largely because such tools appear less frequently in LLMs’ training data.

IV. SYSTEM DESIGN

A. Overview

In this paper, we present CHECKMATE, a system designed to overcome limitations of existing LLM-based pentesting frameworks. Following the PEP diagram proposed in §II-A, CHECKMATE consists of three major components: classical planning+ as the planner, an LLM agent as the executor, and an LLM as the perceptor. The overall design of CHECKMATE is illustrated in Figure 2. Specifically, we introduced predefined attack actions to expand LLM’s knowledge on the specialized tools. Classical planning+ is leveraged to plan the next action, which is executed by an LLM agent. An LLM is used to interpret execution results and update the planner for further planning. Instead of relying on the LLM agent for the entire pentesting workflow, CHECKMATE restricts the LLM’s role to a pure perceptor and a simple-task executor. This design leverages the LLM agent’s strong executing and interpreting capabilities while relieving it of long-horizon planning and reasoning, which are handled by the classical planner.

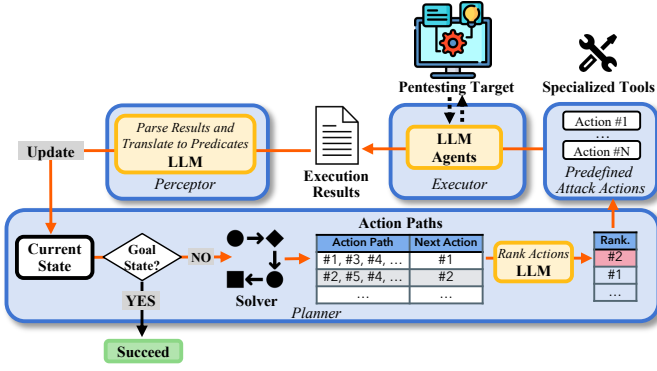


Fig. 2: Overview of CHECKMATE. The orange arrow shows the iterative loop of classical planning+. The current state is initialized before the planning starts.

B. Predefined Attack Actions

As previously mentioned, existing general-purpose LLM agents lack knowledge of specialized tools during pentesting. To address this, we introduce predefined attack actions to expand their knowledge base. We explicitly predefine niche and fine-grained tools such as Metasploit modules, NSE scripts, and Nuclei templates as “actions”, which are considered by the planner. Predefined attack actions also help avoid the inconsistency and errors in LLM-command-generation. In pentesting, most commands adhere to a consistent structure. For example, when executing a default port scan using `nmap -Pn -sC -sV -p- -oN - #{target}`, the command structure remains largely consistent, while the only part that usually changes is `#{target}`. However, the next-token prediction mechanism of LLMs is increasingly unstable, and error-prone when generating long commands. In contrast, predefined attack actions provide the core structure and options of the command, leaving only parameters like `#{target}` to be specified, significantly reducing the risk of generating incorrect commands.

Predefined attack action offers an alternative approach to expanding an LLM’s knowledge base without relying on traditional RAG or fine-tuning. Fine-tuning LLMs is often costly, time-consuming, and difficult to scale. RAG, while flexible, depends on retrieving document snippets and the model’s ability to interpret those snippets and synthesize commands. In contrast, predefined attack actions offer explicit, well-structured, executable commands. By defining preconditions of actions (we will introduce this later), these actions can be retrieved more accurately, efficiently, and interpretably than relying solely on embedding-based similarity search in RAG.

C. Planner

We propose classical planning+ as the planner of CHECKMATE, which encodes the causal relationships explicitly and maintains a persistent and coherent plan throughout the pentesting.

1) *Causal Relationships*: Causal relationships are encoded explicitly through the preconditions and effects associated with

each action. For example, once a web enumeration action identifies a specific web application, a pentester would naturally consider all relevant Metasploit modules, NSE scripts, and Nuclei templates associated with that application. In classical planning, the discovered web application is an effect of the enumeration action and simultaneously serves as a precondition of those subsequent actions. The set of factors used as preconditions and effects is flexible and can be customized or extended as needed. In our current design, these factors include elements such as the identified application, CVEs, URLs, usernames, passwords, etc. By encoding these causal dependencies directly, the system gains stronger built-in causal reasoning capabilities and reduces the need for the LLM to perform complex long-horizon reasoning on its own.

2) *Classical Planning+*: Classical planning+ is proposed to address the limitations of traditional classical planning in dynamic, non-deterministic, and partially observable tasks.

Non-Deterministic Action Effects: Pentesting inherently involves uncertainty and incomplete information. For instance, the result of a port scan is not known until finished, and the outcome of an exploit attempt is often unpredictable until it is executed. However, traditional classical planning assumes a static, deterministic, and fully observable target, where all action effects are determined, and the state of the target is completely specified before the planning starts. Some pentesting systems use complex models to encode uncertainty, which are difficult to scale in the real world. In CHECKMATE, we propose classical planning+, leveraging LLMs to dynamically determine action effects. Since it updates action effects at runtime, complete knowledge is no longer required before planning begins. Specifically, we define the non-deterministic effect to indicate that the effect of an action is unknown until it is executed. Once an action with a non-deterministic effect is executed, LLMs are invoked to analyze the execution outcome and generate concrete effect predicates. We describe this process in §IV-C2, along with a concrete example. Through this mechanism, we successfully extend classical planning to dynamic, non-deterministic, partially-observable scenarios.

Classical planning+ begins from the initial state representing all prior knowledge about the target. In each iteration, the planner checks whether the goal is reachable under the current state. If it is, the planner executes the action sequence leading to the goal. If not, the planner produces a list of applicable actions by checking the preconditions of each action. Next, the LLM executes the optimal action from this list based on its knowledge and updates the initial state. If the executed action has a non-deterministic effect, the LLM is invoked to analyze the execution output and translate it into concrete predicates. This process is repeated iteratively until either the goal is met or all possible actions have been explored. Compared to LLM agents, classical planning+ provides a more structured planning engine by presenting a plan as a directed acyclic graph. It offers several advantages in pentesting planning. First, it exhaustively explores the entire action space, avoiding missing available actions, especially in scenarios with a large number of actions or long action sequences. Second,

it avoids repeating previously executed actions or jumping across different directions, which is a common failure in LLM-based planning. Moreover, the planning process is both visible and interpretable. We illustrate classical planning+ using an example in Figure 3.

Algorithm 1 Iterative Planning for Penetration Testing under Partial Knowledge

```

1: Input: Domain  $D$  with predefined actions, initial knowledge  $I_0$ 
2: Initialize:  $S \leftarrow I_0$  ▷ Initial state from known information (e.g., IP)
3: while termination condition not met and actions remain do
4:   applicableActions  $\leftarrow \{\}$ 
5:   for all action  $a$  in domain  $D$  do
6:     if  $a$  is reachable from state  $S$  then
7:       seq  $\leftarrow \text{plan}(S, a)$ 
8:       applicableActions.add(seq.first())
9:     end if
10:  end for
11:  nextAction  $\leftarrow \text{LLM\_Select}(\text{applicableActions})$ 
12:  result  $\leftarrow \text{Execute}(\text{nextAction})$ 
13:  if nextAction has deterministic effects then
14:     $S \leftarrow S \cup \text{effects}(\text{nextAction})$ 
15:  else
16:    preds  $\leftarrow \text{Parse\_NonDeterministic\_Effects}(\text{result})$ 
17:     $S \leftarrow S \cup \text{preds}$ 
18:  end if
19: end while
20: if goal is not achieved then
21:   Report failure: challenge unsolvable.
22: end if

```

D. Executor

Once the next action is determined, the system should execute it without human intervention. To do so, it must select the appropriate tools, reliably generate precise, executable instructions, and configure all required parameters. Given the strong execution abilities of LLM-based code agents, CHECKMATE employs an LLM agent as its executor. Each predefined action is paired with a concise, action-specific prompt that guides the agent. These prompts specify the required tool and command structure, along with placeholders for parameters. For instance, for a network-scanning action, the prompt clearly outlines the expected flags and arguments, while still allowing task-specific values (the IP address) to be injected. These placeholders are automatically populated by the classical planner, ensuring that critical parameters, such as a module name for an exploitation step, are determined deterministically rather than by the LLM, thereby reducing the risk of hallucinations. After the planner selects an action, CHECKMATE provides the corresponding prompt to the LLM executor, which performs the command and returns the resulting output for downstream processing.

E. Perceptor

The perceptor bridges the executor and the planner: it analyzes the execution results in heterogeneous formats and content, translates them into the representation that the planner can use for subsequent planning. In CHECKMATE, the perceptor translates the outputs into predicates defined in classical planning+, which are then used to update the current state. CHECKMATE has two types of perceptrors: a rule-based perceptor and an LLM-based perceptor. The rule-based perceptor parses structured outputs and maps them to the corresponding predicates, avoiding the randomness introduced when using LLMs. For example, the JSON result returned from a Metasploit search can be directly mapped to a predicate (*msf-module-available ?exploit-name*) for simplicity. The LLM-based perceptor leverages LLMs to interpret unstructured outputs and produce predicates defined in classical planning+.

V. EVALUATION

A. Penetration Capability

We first evaluate the penetration capability of CHECKMATE, compared to existing work. We adopt the same benchmark dataset, metrics, and experimental setup as described in Section 3. The results are shown in Figure 4. CHECKMATE demonstrates substantially stronger penetration capability than all baselines, as evidenced by its progress across all milestones. Notably, 88% of its penetration attempts reach milestone M7, whereas prior work, except Claude Code, rarely progresses beyond M4. Furthermore, CHECKMATE shows advantages over Claude Code at the higher milestones, particularly in the success rates for M6 and M7, indicating improved effectiveness in executing exploits and successfully obtaining a shell. These gains result from the explicitly defined, fine-grained actions and a structured planning strategy. By planning all available actions before committing to any specific attack path, CHECKMATE avoids becoming trapped in unproductive branches and maintains steady progress toward deeper system compromise.

B. Efficiency

In this section, we evaluate both the efficiency and cost of CHECKMATE. We selected 20 penetration tasks that CHECKMATE and Claude Code were both able to successfully complete. Under the same LLM model setting, we compared the total monetary cost, representing the amount of LLM tokens consumed, and the time required to finish each task. The results are summarized in Figure 5. On average, CheckMate has a total cost of \$0.68, which is 53% lower than that of Claude Code under identical conditions. This reduction in token consumption can be attributed to the use of classical planning for strategy formulation. In contrast, Claude Code relies entirely on text-based reasoning, where every intermediate thought and plan must be expressed in natural language, leading to substantial token overhead. By adopting a symbolic and formalized planning mechanism, CHECKMATE avoids using the LLM to “generate” its reasoning process, thereby concentrating the model’s generation capacity on executing

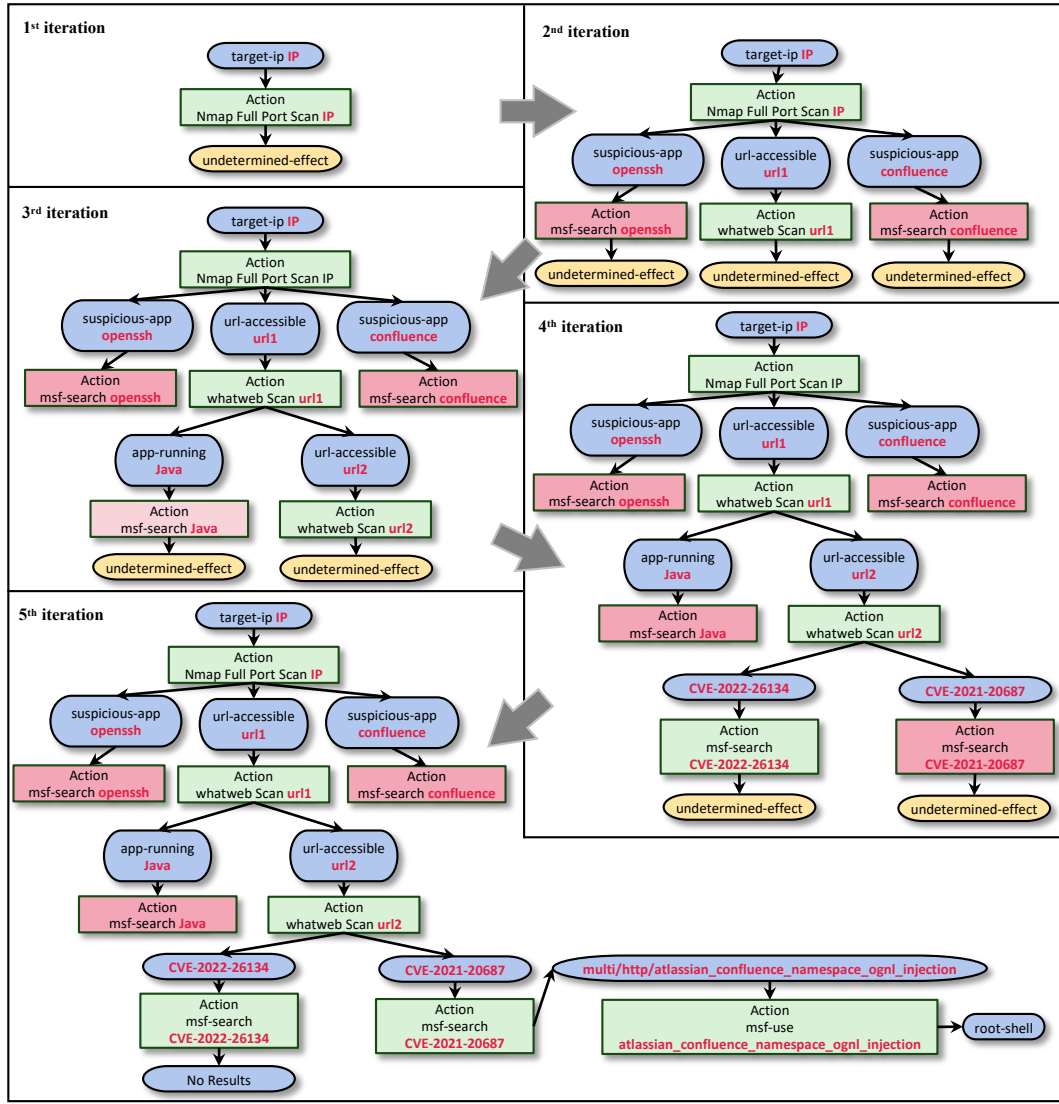


Fig. 3: A pentesting workflow driven by classical planning+. Each panel shows one planning-execution-perception iteration. Blue rounded ovals are predicates that link actions across iterations; yellow rounded ovals denote non-deterministic action effects. Rectangular boxes list feasible actions available during the engagement, and light-green rectangles indicate the actions chosen by the planner for execution in that iteration. Arrows show how actions are connected with predicates.

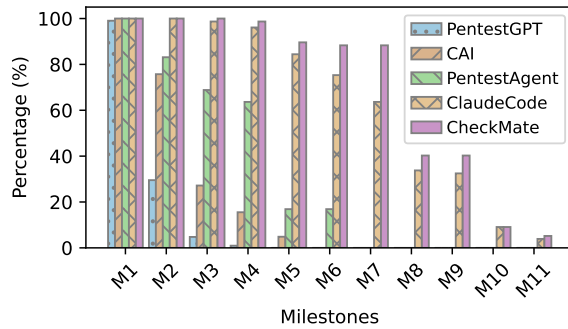
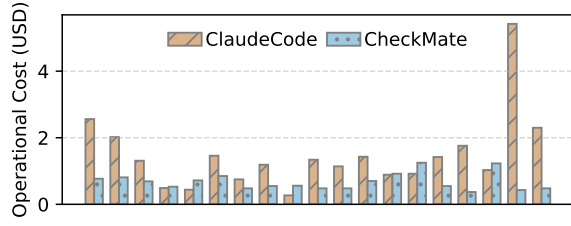


Fig. 4: Comparison of Claude Code with CHECKMATE on Vulhub benchmark

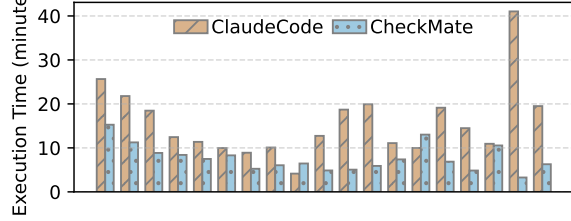
actions and interpreting outputs. The average time consumed for CHECKMATE is 7.75 minutes, which is 54% lower than Claude Code.

C. Stability

In this section, we evaluate the stability of the pentesting process, i.e., whether the system demonstrates consistent performance across repeated executions of the same task. To assess this, we execute each task three times and record the results, costs, and time consumption for each run. We then compute the success rate (i.e., the proportion of runs in which all three attempts successfully achieve penetration) and the *Coefficient of Variation* (a scale-independent measure of dispersion) of both cost and time. The aggregated results are summarized in Table II. About 25% of the tasks



(a) Monetary costs.



(b) Time consumed.

Fig. 5: Efficiency comparison between ClaudeCode and CHECKMATE.

cannot be solved consistently by Claude Code across all three attempts. In addition, CHECKMATE demonstrates higher consistency in both LLM token usage and execution time. These improvements stem from CHECKMATE’s adoption of a more structured planning engine, which reduces unnecessary fluctuations introduced by the LLM.

TABLE II: Stability comparison between CHECKMATE and Claude Code.

	CHECKMATE	Claude Code
Success Rate for all Attempts ($\uparrow$)	100%	75%
Coefficient of Variation - Cost ($\downarrow$)	0.129	0.451
Coefficient of Variation - Time ($\downarrow$)	0.093	0.325

D. Case Study

We analyze a specific example in detail to illustrate differences during pentesting between CHECKMATE and Claude Code. In this case, CHECKMATE completed the penetration in only three steps; Claude Code, by contrast, used 26 steps, many of which were added because of redundancy, premature abandonment, distractions, and incomplete planning and reasoning. The target is an old version of Apache ActiveMQ (an open-source messaging middleware that supports Java messaging services, clustering, and the Spring framework) from Vulhub. CHECKMATE began with a full-port Nmap scan plus fingerprinting and script probes. It discovered two open ports (22 and 8191), identified that Apache ActiveMQ was running, and associated that service with likely CVEs and corresponding Metasploit modules. Rather than rushing straight to exploitation, CHECKMATE chose, from the feasible action set, to analyze the web interface to further confirm the ActiveMQ version. That analysis verified that an ActiveMQ Console was running and revealed the

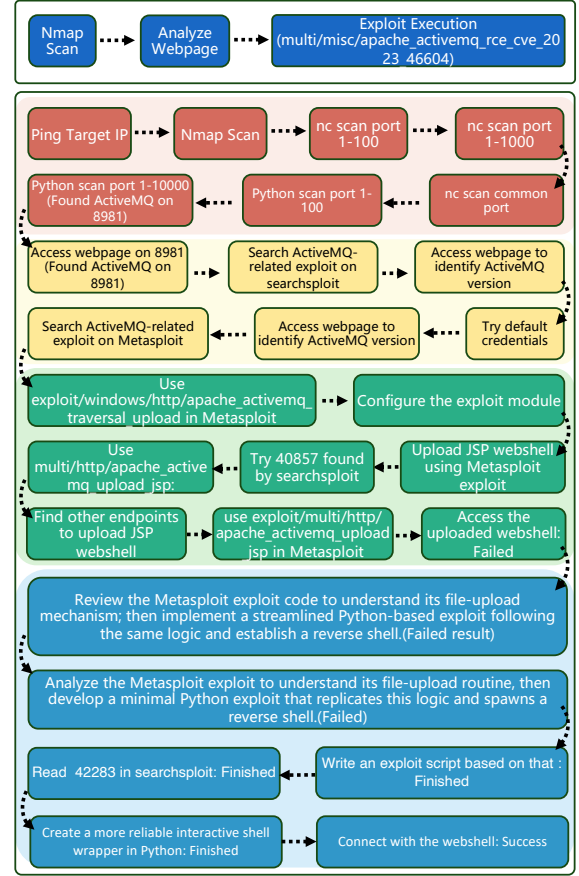


Fig. 6: Top box: CHECKMATE’s workflow. Bottom box: Claude Code’s workflow. Colors show stages. pink: reconnaissance, yellow: search/analysis, green: Metasploit/SearchSploit exploitation (failed), blue: autonomous exploitation.

precise version, 5.11.1. Armed with that version information, in the third step, CHECKMATE selected Metasploit’s “multi/misc/apache_activemq_rce_cve_2023_46604” module and ran the exploit, ultimately obtaining a root shell on the target. At each stage, the LLM evaluated all available actions and prioritized them, demonstrating a highly planned and systematic approach.

By contrast, Claude Code’s test was a largely ad-hoc process, showing exploratory and blind trial-and-error behavior. Claude Code first tried to ping the target IP and run nmap scans, but abandoned both because of insufficient permissions. In fact, both commands failed due to missing socket privileges and would have succeeded with “sudo”. Instead of modifying the commands to add the necessary privileges, Claude Code pivoted to using Netcat and writing Python scripts, which are more complex methods for port scanning. Its port-scanning also lacked a coherent plan: it scanned the first 100 ports, then 1,000 ports, then “common” ports, and only later broadened the range, thus finally finding port 8191 open. It is a risky strategy because when Claude Code hit a rabbit hole on common ports, it tended to pursue the wrong path, wasting